

DS 246: Final Project Presentation

Federated Fine-Tuning of LLMs

Capstone Project: Phase 2 Submission

Moving from Centralized to Distributed Training

Tej Pratap Yadav & P Sai Aditya

The Challenge: Data & Compute

Data Privacy & Silos

Sensitive Data (such as Medical) is highly sensitive and protected by regulations. Consolidating this data into a centralized server for fine-tuning LLMs poses significant privacy risks and logistical challenges.

Resource Constraints

Fine-tuning Large Language Models (LLMs) requires massive GPU memory. Our initial attempts with Mistral highlighted the difficulty of training substantial models on limited hardware resources.

Goal

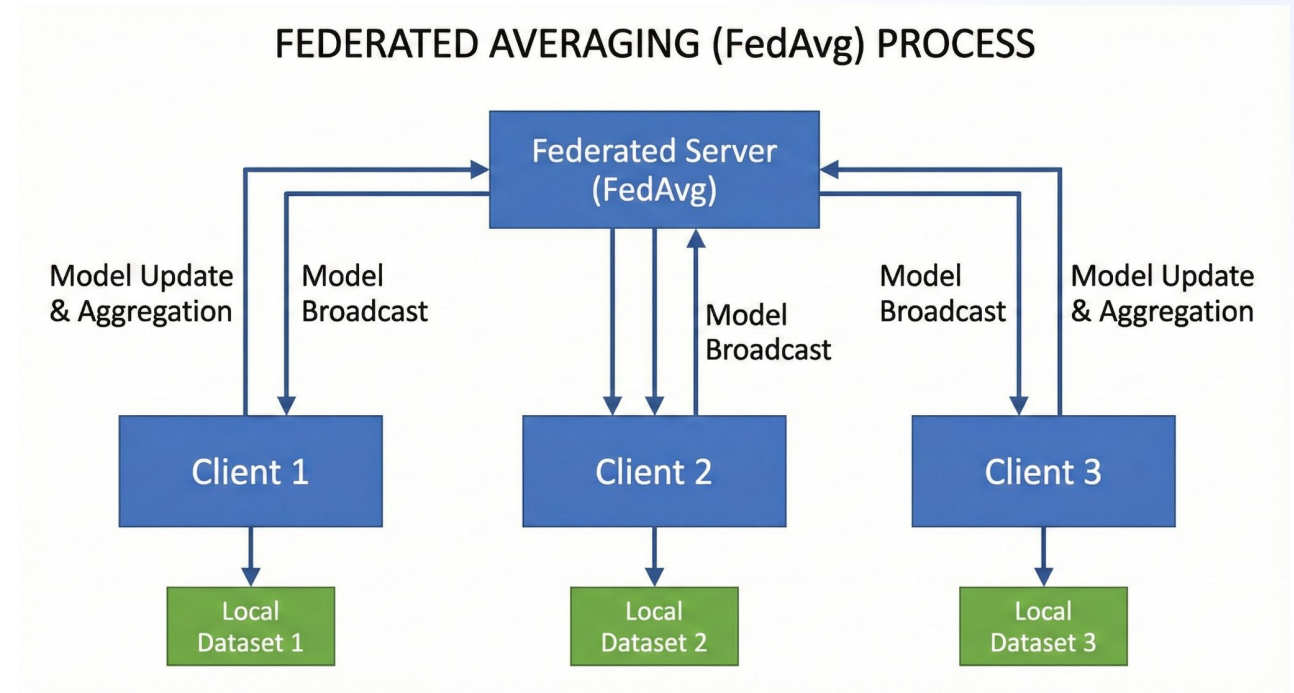
Federated Learning Strategy

We propose a Federated Learning pipeline using the **FedAvg** algorithm. Instead of moving data, we move the model.

Clients train locally on their private data partitions (PDF/JSONL) and send only the model weight updates (LoRA deltas) to a central server for aggregation.

$$\mathbf{w}_{t+1} = \sum_{k=1}^K \frac{n_k}{n} \mathbf{w}_{t+1}^k$$

Global Model Aggregation Formula



End-to-End Workflow

1. Baseline

Centralized
fine-tuning of the
base LLM on MedQA
to establish reference
performance metrics.

2. Distribution

Data is split into
separate "clients" to
simulate a distributed
environment with
private local datasets.

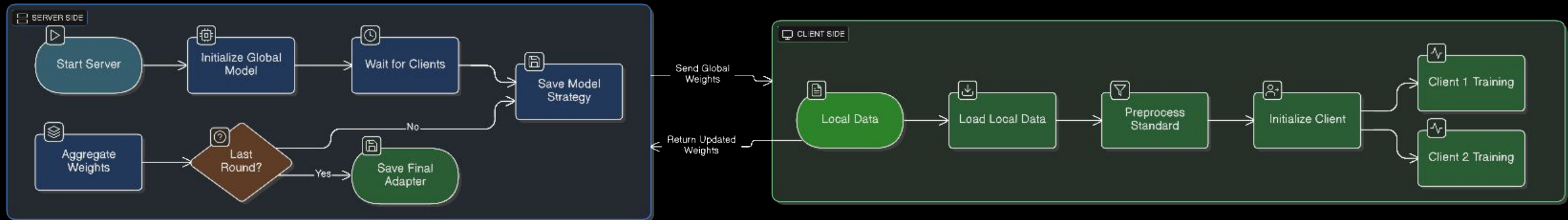
3. Aggregation

Clients train locally.
LoRA updates are sent
to the server and
aggregated using
FedAvg.

4. Evaluation

Comparison of
Federated vs.
Centralized models on
accuracy and privacy
preservation.

End-to-End WorkFlow



Key System Features



Dual Format Support

Seamlessly handles **PDF** files for chunk-based text generation and **JSONL** for structured instruction fine-tuning.



Adaptive Model Usage

Support for switching base models. Transitioned from Mistral to **Qwen** to optimize performance under GPU constraints.



Dynamic Client Control

Interactive slider to configure the number of federated clients (simulated) participating in the training round.

System Architecture

Streamlit UI Interface → Orchestration Server → Distributed Clients

Interactive UI Demo

Streamlit Implementation

We have developed a functional UI that allows users to configure the entire training process.

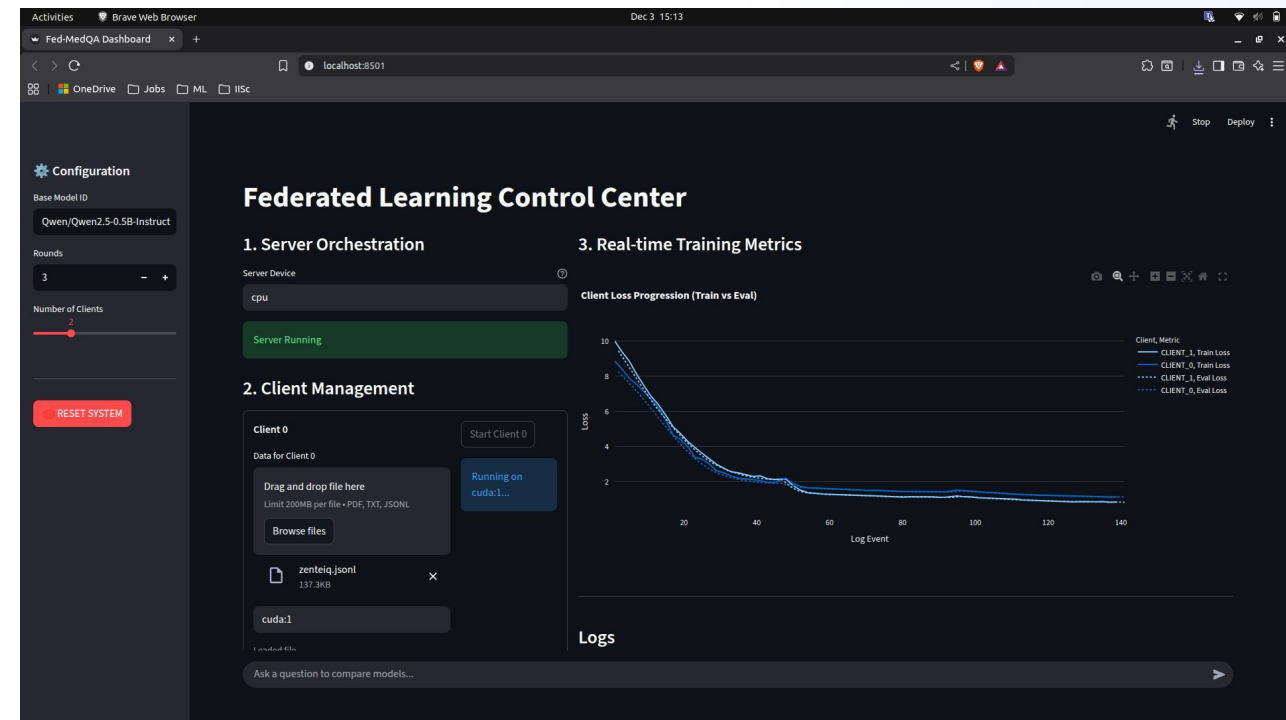
Step 1: Select Base Model (e.g., Qwen, Mistral).

Step 2: Upload Dataset. The system auto-detects format:

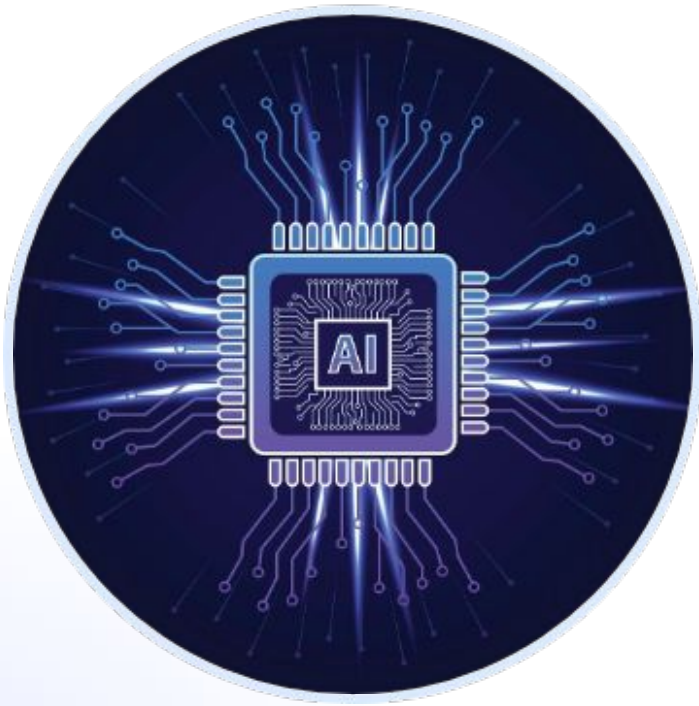
- *PDF*: Unstructured text generation.
- *JSONL*: Instruction tuning (Q&A pairs).

Step 3: Configure Client Count (Slider).

Step 4: Visualise Training & Evaluate.



Model Evolution



Why Switch to Qwen?

Our journey began with smaller models like TinyGPT2 and TinyLlama, which failed to yield sensible results on the MedQA dataset.

We successfully implemented **Mistral-7B**, but realized that federated fine-tuning multiple instances of such a large model exceeded our available GPU memory.

Qwen was selected as the optimal balance between performance and computational efficiency for our federated demonstration.

Preliminary Results (Mistral Baseline)

Before switching, we benchmarked the centralized fine-tuning of Mistral-7B on the MedQA dataset (1272 test questions).



Fine-tuning yielded a **~5% improvement** in accuracy.

Phase 2

-  **Qwen Centralized Baseline:** Re-run the MedQA benchmark on the Qwen base model to establish a new reference point.
-  **Federated Qwen Implementation:** Execute the full federated pipeline with Qwen, aggregating weights from distributed clients.
-  **Comparative Evaluation:** Populate the final results table comparing Centralized vs. Federated accuracy for Qwen.
-  **Differential Privacy:** Introduce gradient clipping and noise addition to the federated updates to enhance data privacy.

Flower Protocol: Weight Transmission

1. Communication (gRPC)

- ⚡ Flower utilizes **gRPC** (Google Remote Procedure Call) over HTTP/2 for efficient, language-agnostic communication between clients and server.
- 📦 Model weights (NumPy arrays) are converted into highly compressed binary **Protocol Buffers (Protobuf)** for transport.
- 🔗 Remote IP addresses work seamlessly as they are simply network endpoints. The Client initiates the connection to the Server's IP:Port.

2. Data Flow Pipeline

➡ Client <-> Server (Weights Update)

1. **Serialization:** NumPy Arrays (Weights) are converted to a compressed byte stream (Protobuf).
2. **Transmission:** Byte stream is sent over the network (IP:Port).
3. **Deserialization:** Server/Client unpacks Protobuf message and converts back to NumPy arrays for aggregation.

Note: Firewalls must permit traffic on the designated port for remote connections to succeed.

Federated Fine-tuning Results

Model – Qwen/Qwen2.5-0.5B-Instruct
Dataset – bigbio/med_qa

	Base Model	Centralized Fine-Tuned	Federated Fine-Tuned
Total Questions	1272	1272	1272
Correct Answer	351	325	358
Accuracy	27.59%	25.55%	28.14%

sshleifer/tiny-gpt2 - 2 Clients - 3 Rounds



sshleifer/tiny-gpt2 - Base Model vs Federated



Base Model (Pre-trained)

Without Federated Fine-tuning

Global Model (Federated)

Aggregated knowledge from all clients

[illegible]

Ask a question to compare models...



Qwen2.5-0.5B-Instruct - 2 Clients - 3 Rounds

3. Real-time Training Metrics

Client Loss Progression (Train vs Eval)



Qwen2.5-0.5B-Instruct – 2 Clients – 3 Rounds



What are partial differential equations (PDEs) used for in the context of AiREX lab's research?

Base Model (Pre-trained)

Without Federated Fine-tuning

A Partial Differential Equation (PDE) is a type of mathematical equation that describes how a system of variables changes over space

Global Model (Federated)

Aggregated knowledge from all clients

Partial Differential Equations (PDEs) are mathematical equations that describe non-differential phenomena, such as heat transfer, wave propagation, and fluid dynamics. In the context of AiREX lab's research, PDEs are often used to model complex systems where the behavior is influenced by multiple variables or parameters simultaneously. This allows researchers to capture the underlying physical processes more accurately than using single-variable models alone. For example, in fluid dynamics, PDEs can be used to describe the flow of fluids through porous media, while in electrical circuit analysis, PDEs can model the behavior of electric currents in various geometries. By solving PDEs numerically, researchers can obtain accurate solutions to these complex systems, which can then be used to predict and optimize real-world applications.

Ask a question to compare models...



Individual Contributions

Tej Pratap Yadav

Lead on Fine-Tuning

Responsible for fine-tuning the models.. Research on models to be used for demonstration on limited GPU resources.

P Sai Aditya

Lead on UI integration.

Focusing on the UI Integration and Dataset Pipeline creation for PDF, JSON, Research on Federated Learning frameworks to be used.

Questions?

Thank you for your time.

Github Link Of the Project

https://github.com/tyadav4268/federated_finetuning_llm

Image Sources



https://pub.mdpi-res.com/electronics/electronics-11-00670/article_deploy/html/images/electronics-11-00670-g001.png?1645607617

Source: www.mdpi.com



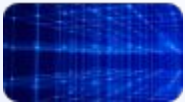
<https://cdn.dribbble.com/userupload/15813489/file/original-9784c3910b8ae034233dc1730426afbd.jpeg?resize=400x0>

Source: dribbble.com



https://static.vecteezy.com/system/resources/previews/068/023/281/non_2x/brightly-glowing-ai-chip-on-light-circuit-board-on-technology-blue-background-cpu-processor-or-semiconductor-on-tech-background-computer-microchip-on-motherboard-vector.jpg

Source: www.vecteezy.com



https://img.freepik.com/premium-photo/block-chain-technology-concept-3d-rendering-network-connection-structure-big-data-visualization_105800-2024.jpg?semt=ais_hybrid&w=740&q=80

Source: www.freepik.com